

ClinicaApp

Sistema de Gestión de Turnos Médicos — Java OOP

Java · OOP · Persistencia CSV	Equipos: 3 a 4 participantes	Entrega: Repositorio público en GitHub
-------------------------------	------------------------------	--

Construirás desde cero un sistema funcional que gestiona pacientes, médicos y turnos de una clínica. El sistema mantiene los datos en memoria durante la sesión y los guarda en archivos CSV al salir, de modo que persisten entre ejecuciones. Al terminar tendrás una aplicación Java de consola que modela entidades reales, aplica validaciones, implementa interfaces y demuestra los principios de la programación orientada a objetos.

1. Cómo funciona el sistema

El sistema usa un modelo en memoria con persistencia al salir. Al arrancar, **DatosCSV.cargar()** lee los archivos CSV y reconstruye los objetos Java en tres listas. Durante la sesión todas las consultas y validaciones operan sobre esas listas. Al salir, **DatosCSV.guardar()** serializa las listas de nuevo a CSV para que los datos sobrevivan al próximo arranque.

```
// FLUJO COMPLETO DEL PROGRAMA

// Al arrancar:
1. Main llama a DatosCSV.cargar(servicio)
   → lee pacientes.csv → llena List<Paciente>
   → lee medicos.csv   → llena List<Medico>
   → lee turnos.csv    → llena List<Turno>
2. Muestra el menú en bucle

// Durante la sesión:
→ Todas las operaciones validan y modifican las listas en memoria

// Al salir (opción 0):
→ Main llama a DatosCSV.guardar(servicio)
  → escribe pacientes.csv, medicos.csv, turnos.csv
→ Imprime mensaje de despedida
```

Los archivos CSV se crean automáticamente en la primera ejecución si no existen. No necesitas crearlos manualmente.

2. Estructura del proyecto

Crea un proyecto Java normal en IntelliJ IDEA (sin Maven, sin frameworks). Organiza los archivos exactamente como se muestra. Cada paquete tiene una responsabilidad única y separada.

```
clínicaapp/
├── src/
│   ├── co/generation/clinica/
│   │   ├── model/
│   │   │   ├── Paciente.java
│   │   │   ├── Medico.java
│   │   │   ├── Turno.java
│   │   │   ├── EstadoTurno.java ← enum
│   │   │   ├── Especialidad.java ← enum
│   │   │   ├── interfaces/
│   │   │   │   ├── Registrable.java
│   │   │   │   ├── Consultable.java
│   │   │   ├── service/
│   │   │   │   ├── ClinicaService.java
│   │   │   ├── datos/
│   │   │   │   ├── DatosCSV.java ← código completo en sección 8
│   │   │   │   ├── Main.java
```

Paciente y **Medico** implementan **Registrable**. **Turno** contiene referencias a objetos **Paciente** y **Medico** completos — no sus ids. **ClinicaService** implementa **Consultable** y contiene toda la lógica de negocio. **DatosCSV** se encarga exclusivamente de leer y escribir los archivos. **Main** solo crea el servicio, carga datos y muestra el menú.

3. Modelo de datos — las tres entidades

Cada clase del paquete **model** representa una entidad del negocio. Sus atributos son privados y solo se acceden mediante getters y setters. Los setters validan antes de asignar — si el valor no es válido lanzan **IllegalArgumentException** con un mensaje descriptivo.

Paciente

Atributo	Tipo	Restricción
id	int	Identificador único. Se asigna en ClinicaService al registrar, no en el constructor.
cedula	String	No puede ser nula ni vacía. Es el criterio de unicidad — no pueden existir dos pacientes con la misma cédula.
nombre	String	No puede ser nulo ni vacío. Aplica trim() antes de asignar.
apellido	String	No puede ser nulo ni vacío. Aplica trim() antes de asignar.
telefono	String	Solo dígitos, entre 7 y 10 caracteres. Validar con expresión regular: <code>^[0-9]{7,10}\$</code>

Dos constructores requeridos: uno que recibe el id (para reconstruir desde CSV) y uno sin id (para registrar desde el menú, donde el id lo asigna ClinicaService). Este patrón se repite en Medico y Turno.

equals() en Paciente: dos pacientes son iguales si y solo si tienen la misma cédula. Sin este método, ClinicaService no podría detectar duplicados con contains() — Java compararía direcciones de memoria, no datos.

toString(): formato esperado → "María García - 1020304050 - 3001234567" (nombre + apellido + cédula + teléfono separados por guión).

Medico

Atributo	Tipo	Restricción
id	int	Identificador único. Se asigna en ClinicaService.
nombre	String	No puede ser nulo ni vacío. Aplica trim().
apellido	String	No puede ser nulo ni vacío. Aplica trim().
especialidad	Especialidad	Una de las cuatro especialidades del enum. No puede ser nula.

equals() en Medico: dos médicos son iguales si tienen el mismo nombre y apellido sin distinguir mayúsculas. Usa equalsIgnoreCase() en ambos campos.

toString(): formato esperado → "Dr. Carlos Pérez - CARDIOLOGIA".

Turno

Atributo	Tipo	Restricción
id	int	Identificador único. Se asigna en ClinicaService.
paciente	Paciente	Referencia al objeto Paciente completo. No puede ser nulo.
medico	Medico	Referencia al objeto Medico completo. No puede ser nulo.
fechaHora	LocalDateTime	Fecha y hora del turno. No puede ser nula. Importar desde java.time.
estado	EstadoTurno	Estado actual. Al crearse siempre inicia en EstadoTurno.PENDIENTE.

equals() en Turno: dos turnos son iguales si tienen el mismo médico y la misma fechaHora. Esto permite detectar conflictos de agenda: un médico no puede tener dos turnos en el mismo momento.

toString(): formato esperado → "[PENDIENTE] María García — Dr. Carlos Pérez (CARDIOLOGIA) — 2026-06-10T09:30".

4. Enumeraciones (paquete model)

¿Por qué enum y no String? Si guardas la especialidad como String, alguien podría escribir "Pediatria", otro "PEDIATRIA", otro "pediatría" — tres valores distintos para Java que representan lo mismo. Un enum define exactamente los valores posibles; el compilador rechaza cualquier otro. Es más seguro, más claro y más fácil de mantener.

Enum	Valores	Dónde se usa
EstadoTurno	PENDIENTE · ATENDIDO · CANCELADO	Atributo estado en Turno. Un turno nace siempre como PENDIENTE.
Especialidad	GENERAL · PEDIATRIA · CARDIOLOGIA · URGENCIAS	Atributo especialidad en Medico. Solo se pueden registrar médicos con una de estas cuatro especialidades.

5. Interfaces (paquete interfaces)

Una interfaz es un contrato. Define qué métodos debe tener una clase sin decir cómo los implementa. Al escribir **implements Registrable** en Paciente o Medico, Java exige que esa clase tenga exactamente esos métodos — si no, el compilador falla.

Registrable

La implementan Paciente y Medico.

Método	Qué retorna
String getDatosRegistro()	Un String con los datos del objeto listos para mostrar en consola. Puede reutilizar toString() o construirlo de forma independiente.
boolean esValido()	true si todos los atributos obligatorios están correctamente definidos (no nulos, no vacíos, formatos válidos). ClinicaService llama a este método antes de registrar cualquier entidad.

Consultable

La implementa ClinicaService. Todos los métodos operan sobre las listas en memoria.

Método	Qué retorna
List listarTurnosDelDia(LocalDate fecha)	Todos los turnos cuya fechaHora corresponda a ese día, ordenados de menor a mayor por hora. Lista vacía si no hay ninguno.
List buscarPorMedico(Medico medico)	Todos los turnos asignados al médico indicado. Usa el equals() de Medico. Lista vacía si no hay ninguno.
List buscarPorPaciente(Paciente paciente)	Todos los turnos del paciente indicado. Usa el equals() de Paciente. Lista vacía si no hay ninguno.

6. ClinicaService (paquete service)

Es el núcleo del sistema. Implementa Consultable y contiene toda la lógica de negocio. Declara tres listas privadas que DatosCSV llena al arrancar y que se mantienen sincronizadas durante la sesión. Los métodos de Consultable operan exclusivamente sobre esas listas — no leen archivos.

Las tres listas deben ser accesibles para DatosCSV. Declara getters para cada una: getPacientes(), getMedicos(), getTurnos(). DatosCSV las usará para guardar y para agregar los objetos reconstruidos desde CSV.

Métodos de Paciente

Método	Lógica esperada
void registrarPaciente(Paciente p)	1. Llama a p.esValido() — si retorna false, imprime error y sale. 2. Verifica que no exista otro paciente con la misma cédula (usa contains()). 3. Asigna el id: toma el máximo id existente en la lista y suma 1 (o usa 1 si la lista está vacía). 4. Agrega p a la lista. 5. Imprime mensaje de éxito con los datos del paciente.
Paciente buscarPorCedula(String cedula)	Recorre la lista con un for. Retorna el Paciente cuya cédula coincida exactamente. Retorna null si no encuentra ninguno. No imprime nada.
void listarPacientes()	Ordena una copia de la lista por apellido y luego por nombre usando Comparator. Imprime cada paciente con toString(). Si la lista está vacía, imprime un mensaje informativo.

Métodos de Médico

Método	Lógica esperada
void registrarMedico(Medico m)	1. Llama a m.esValido(). 2. Verifica duplicado con contains() — invoca el equals() de Medico. 3. Asigna el id igual que en registrarPaciente. 4. Agrega a la lista. Imprime éxito.
Medico buscarPorNombreApellido(String nombre, String apellido)	Recorre la lista. Retorna el Medico cuyo nombre y apellido coincidan sin distinguir mayúsculas (equalsIgnoreCase). Retorna null si no existe.
void listarMedicos()	Ordena una copia por especialidad (orden natural del enum) y luego por apellido. Imprime cada médico con toString(). Si la lista está vacía, imprime un mensaje informativo.

Métodos de Turno

Método	Lógica esperada
void asignarTurno(Turno t)	1. Verifica que el paciente del turno exista en la lista (busca por cédula). 2. Verifica que el médico del turno exista (busca por nombre y apellido). 3. Verifica que no exista ya un turno con el mismo médico y fechaHora (usa contains()). 4. Asigna el id. Agrega a la lista. Imprime éxito con toString().
void cancelarTurno(int idTurno)	Busca el turno por id. Si no existe: imprime 'Turno no encontrado.' Si existe pero su estado es ATENDIDO o CANCELADO: imprime que no se puede cancelar. Si está PENDIENTE: cambia el estado a CANCELADO con setEstado(). Imprime confirmación.

<code>void cambiarEstadoTurno (int idTurno, EstadoTurno nuevo)</code>	Busca el turno por id. Si no existe: imprime 'Turno no encontrado.' Si existe: cambia el estado con <code>setEstado()</code> e imprime confirmación con el nuevo estado.
---	---

<code>void cambiarEstadoTurno (int idTurno, EstadoTurno nuevo)</code>	Busca el turno por id. Si no existe: imprime 'Turno no encontrado.' Si existe: cambia el estado con <code>setEstado()</code> e imprime confirmación con el nuevo estado.
---	---

7. Main.java — punto de entrada

Main tiene una única responsabilidad: crear el `ClinicaService`, cargar los datos, mostrar el menú en bucle y guardar al salir. No debe contener lógica de negocio ni validaciones — todo eso va en `ClinicaService`.

7. Main.java — punto de entrada

Main tiene una única responsabilidad: crear el `ClinicaService`, cargar los datos, mostrar el menú en bucle y guardar al salir. No debe contener lógica de negocio ni validaciones — todo eso va en `ClinicaService`.

```
// Secuencia en main():
ClinicaService servicio = new ClinicaService();
DatosCSV.cargar(servicio);

// bucle while con Scanner para el menú...

// Al salir (opción 0):
DatosCSV.guardar(servicio);
System.out.println("Hasta pronto. Datos guardados.");
```

Menú esperado en consola

[illegible]

Para la opción 3 (asignar turno): pide la cédula del paciente y el nombre + apellido del médico, búscalos en `ClinicaService`, y si ambos existen construye el `Turno` con esos objetos. Para la fecha y hora pide año, mes, día, hora y minuto por separado y construye el `LocalDateTime` con `LocalDateTime.of(año, mes, día, hora, minuto)`.

8. DatosCSV.java — código completo

Este es el único archivo que recibes resuelto. Cópialo exactamente en el paquete **datos** de tu proyecto. No necesitas modificarlo — solo llamar a sus dos métodos estáticos desde Main.

DatosCSV crea los archivos CSV si no existen. En la primera ejecución el sistema arrancará con las listas vacías y los archivos se generarán al salir con los datos que hayas ingresado durante la sesión.

```
package co.generation.clinica.datos;
import co.generation.clinica.model.*;
import co.generation.clinica.service.ClinicaService;
import java.io.*; import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter; import java.util.List;

public class DatosCSV {
    private static final String DIR = "datos/";
    private static final String F_PACIENTES = DIR + "pacientes.csv";
    private static final String F_MEDICOS = DIR + "medicos.csv";
    private static final String F_TURNOS = DIR + "turnos.csv";
    private static final DateTimeFormatter FMT =
        DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm");

    public static void cargar(ClinicaService servicio) {
        new File(DIR).mkdirs();
        cargarPacientes(servicio); cargarMedicos(servicio); cargarTurnos(servicio);
    }

    private static void cargarPacientes(ClinicaService servicio) {
        File f = new File(F_PACIENTES); if (!f.exists()) return;
        try (BufferedReader br = new BufferedReader(new FileReader(f))) {
            String linea;
            while ((linea = br.readLine()) != null) {
                if (linea.isBlank()) continue;
                String[] p = linea.split(",", -1); // id,cedula,nombre,apellido,telefon
                servicio.getPacientes().add(new Paciente(
                    Integer.parseInt(p[0].trim()),
                    p[1].trim(), p[2].trim(), p[3].trim(), p[4].trim()));
            }
        } catch (IOException e) { System.out.println("Error: " + e.getMessage()); }
    }
}
```

```

private static void cargarMedicos(ClinicaService servicio) {
    File f = new File(F_MEDICOS); if (!f.exists()) return;
    try (BufferedReader br = new BufferedReader(new FileReader(f))) {
        String linea;
        while ((linea = br.readLine()) != null) {
            if (linea.isBlank()) continue;
            String[] p = linea.split(",", -1); // id,nombre,apellido,especialidad
            servicio.getMedicos().add(new Medico(
                Integer.parseInt(p[0].trim()), p[1].trim(), p[2].trim(),
                Especialidad.valueOf(p[3].trim())));
        }
    } catch (IOException e) { System.out.println("Error: " + e.getMessage()); }
}

private static void cargarTurnos(ClinicaService servicio) {
    File f = new File(F_TURNOS); if (!f.exists()) return;
    try (BufferedReader br = new BufferedReader(new FileReader(f))) {
        String linea;
        while ((linea = br.readLine()) != null) {
            if (linea.isBlank()) continue;
            // formato: id,cedulaPaciente,nombreMedico,apellidoMedico,fechaHora,estado
            String[] p = linea.split(",", -1);
            Paciente pac = servicio.buscarPorCedula(p[1].trim());
            Medico med = servicio.buscarPorNombreApellido(
                p[2].trim(), p[3].trim());
            if (pac == null || med == null) continue;
            servicio.getTurnos().add(new Turno(
                Integer.parseInt(p[0].trim()), pac, med,
                LocalDateTime.parse(p[4].trim(), FMT),
                EstadoTurno.valueOf(p[5].trim())));
        }
    } catch (IOException e) { System.out.println("Error: " + e.getMessage()); }
}

```


[illegible]

9. Criterios de evaluación

Criterio	Qué se evalúa
Estructura	Paquetes organizados según la sección 2. Separación clara de responsabilidades entre model, interfaces, service y datos. Nombres de clases, métodos y variables descriptivos.

Funcionalidad	El sistema arranca y carga datos correctamente si ya existen archivos CSV. Las 10 opciones del menú operan sin errores. Las validaciones funcionan: sin duplicados, sin conflictos de agenda, estados actualizados correctamente. Los datos persisten entre ejecuciones (verificable cerrando y reabriendo el programa).
Calidad OOP	Las interfaces están implementadas correctamente en todas las clases que corresponde. Los enums se usan donde corresponde — nunca un String para estado o especialidad. Los equals() están definidos y funcionan. Los setters validan. El código es limpio: sin lógica duplicada, sin comentarios obsoletos.

10. Entrega

El repositorio debe ser **público en GitHub** y contener el proyecto Java completo. El **README.md** debe incluir: nombre del equipo, integrantes, e instrucciones para correr el proyecto en IntelliJ. El link del repositorio se sube en Canvas al finalizar.

Recuerda: los archivos CSV se generan en una carpeta **datos/** dentro del directorio desde donde ejecutas el programa. En IntelliJ, eso es la raíz del proyecto. Puedes agregarlos al `.gitignore` si no quieres subir datos de prueba al repositorio.